



**Universitat
de Lleida**

HPC Project:
Heat Diffussion Equation
High Performance Computing

**High Performance Computing:
Delivery 2:
Hybrid Implementation with OpenMP-MPI**

Gerard Jaimejuan Guardia
Yanis Amrane
18/05/2025

1. Introduction

The objective of this second delivery is to improve the heat diffusion program by implementing a hybrid parallelization using both MPI (to distribute the grid among processes) and OpenMP (to parallelize internal computations). The aim is to evaluate hybrid scalability as a function of problem size and the number of cores used.

2. Parallelization Decisions

2.1. Chosen Strategy

We opted for a **horizontal strip decomposition**, where each MPI process handles a sub-block of rows of the grid. **Ghost rows** are used to exchange boundary values with neighboring processes.

Within each process, internal computations are parallelized using **OpenMP** (with *#pragma omp parallel for collapse(2)* on the *i/j* loops).

2.2. Data and Communication Management

- **Initialization:** performed in process 0, then distributed via `MPI_Scatterv`
- **Ghost row exchange:** done using `MPI_Sendrecv` between neighbors
- **Final gathering:** the full grid is reassembled via `MPI_Gatherv` and written by process 0 into a BMP file

2.3. Dynamic Output Naming

Output files are dynamically named to clearly reflect the configuration, e.g.:

output_hybrid_1000_1000_mpi2_omp2.bmp

3. Performance and Scalability

3.1. Methodology

- Three grid sizes tested: 100x100, 1000x1000, 2000x2000
- Four configurations:
 - MPI=1, OMP=1 (sequential reference)
 - MPI=1, OMP=4 (OpenMP only)

- MPI=2, OMP=2 (hybrid)
- MPI=4, OMP=1 (MPI only)
- Execution times were measured using `MPI_Wtime()` on rank 0
- Speedup was computed as T_{ref} / T_{test} with T_{ref} = time with 1 MPI, 1 thread for the same grid and T_{test} = Time with X MPI procs × Y threads
- Efficiency as $speedup / total_cores * 100$ with $total_cores = nb_mpi \times nb_threads$

3.2. Experimental Results

Grid Size	Steps	MPI Procs	OMP Threads	Total Cores	Time (s)	Speed up	Efficiency (%)
100x100	1000	1	1	1	0.060525	1.00	100
100x100	1000	1	4	4	0.019211	3.15	78.75
100x100	1000	2	2	4	0.019098	3.17	79.25
100x100	1000	4	1	4	0.016880	3.59	89.75
1000x1000	1000	1	1	1	4.996376	1.00	100
1000x1000	1000	1	4	4	1.558956	3.20	80

1000x1000	1000	2	2	4	1.581283	3.16	79
1000x1000	1000	4	1	4	1.585208	3.15	78.75
2000x2000	10000	1	1	1	195.056040	1.00	100
2000x2000	10000	1	4	4	61.663589	3.16	79
2000x2000	10000	2	2	4	61.955766	3.15	78.75
2000x2000	10000	4	1	4	61.749036	3.16	79

3.3. Interpretation

Across all tested grid sizes, we observe consistent and meaningful speedups with increased parallel resources:

- For the smallest grid 100x100, speedup reaches up to **3.59x** (MPI=4, OMP=1), demonstrating good parallel responsiveness even on light workloads. Efficiency peaks at **89.75%**, confirming that overhead is minimal in this case.
- For the medium-sized grid 1000x1000, speedups remain consistent across configurations, with all parallel setups achieving **~3.15x to 3.20x** improvement over the serial case. The hybrid and pure MPI versions perform comparably, and efficiency stays close to **80%**, showing balanced load distribution.
- For the largest grid 2000x2000, the same trend holds. Despite the increased workload, all parallel configurations achieve a speedup of **~3.15x**, again with efficiency values ranging from **78% to 79%**. This confirms that the hybrid implementation scales well with problem size.

Notably, no single configuration significantly outperforms the others, which demonstrates that both OpenMP and MPI contribute effectively to performance. The MPI=2, OMP=2 case consistently achieves high speedup and efficiency, suggesting it is a balanced configuration for 4-core execution.

Because each node in the cluster has only four physical cores, you hit a natural limit as soon as those four cores are fully occupied. Requesting more MPI ranks or OpenMP threads doesn't give you extra hardware, it just makes the same cores spend extra time coordinating, and if the job spills onto a second node the ranks must exchange data over the network, which is slower still. That is why every 4-way setup (whether 4 MPI, 4 OpenMP, or the 2 + 2 hybrid) have roughly the same speed-up, and why the 2 MPI + 2 OpenMP mix looks slightly better: it fills the four cores cleanly while keeping communication overhead to a minimum.

4. Visualization

The output BMP files have been generated and can be viewed in the Github repository (<https://github.com/High-Performance-Computing-MEINF/heat-diffusion-hybrid>), in the out/ directory. These help visually confirm that the simulation results are consistent across configurations.

5. Conclusion

The hybrid parallelization leads to measurable performance gains, provided the problem size is large enough. The code is stable and reusable. For even larger problems, pure MPI or adaptive hybrid configurations (based on system topology) could be considered.